

RAP 後端開發練習 1：為什麼用 RAP？環境限制聲明

Lecture

從「寫一個 OData Service」的角度看，過去的做法有什麼問題

前面的課程已經教過幾種讓外部系統存取 SAP 資料/邏輯的方式：REST 課程 ([src/ABAP_Training_REST/](#)) 教的是自己寫 [CL_REST_HTTP_HANDLER](#) 子類手刻一個 HTTP Service；AMDP 課程教的是把運算邏輯下推到資料庫層；更早的基礎課則是傳統的 Function Module / BAPI，靠 RFC 或手動包裝暴露給外部呼叫。這幾種做法有一個共同的痛點：**CRUD (新增/查詢/修改/刪除)**、**鎖定**、**權限檢查**、**資料完整性驗證**這些「每個資料物件幾乎都要重寫一遍」的邏輯，全部要自己手動實作——自己寫 [ENQUEUE/DEQUEUE](#)、自己寫 [AUTHORITY-CHECK](#)、自己判斷這次是 Insert 還是 Update、自己維護 OData 的 [\\$metadata](#) 描述.....工程量大，而且每個開發者寫出來的風格都不一樣，維護起來很痛苦。

RAP (ABAP RESTful Application Programming Model) 要解決的正是這個問題：你只要用 CDS (Core Data Services) 宣告資料模型、用 Behavior Definition 宣告 (不是用程序式邏輯寫) 這個資料物件支援哪些操作 (能不能 Create / Update / Delete)、有哪些自動衍生欄位 (Determination)、哪些檢查規則 (Validation)、哪些自訂操作 (Action)，框架會自動生成鎖定管理、草稿處理 (如果用到)、OData 協定轉換這些「重複性高、容易寫錯」的基礎設施代碼，開發者只需要專注在**業務邏輯**本身。這是 SAP 現在新開發案的官方建議架構，取代舊式的「Screen + FM + 手寫商業邏輯」模式 (俗稱 SAP GUI Programming Model)，也是 Fiori Elements 這類「畫面自動生成」前端技術背後的資料服務標準供應方式。

本課程的範圍：只教到「OData Service 正確、能被呼叫、資料進得去出得來」為止——資料模型設計、CRUD、Determinations、Validations、Actions、Service 發布。因為要銜接後續規劃中的 **Fiori Elements** 課程，rap02 會額外教 CDS View 的 ****@UI.*** Annotation 語法 (**Metadata Extension / DDLX** 物件怎麼建、常用的 [@UI.headerInfo](#) / [@UI.lineItem](#) / [@UI.selectionField](#) 怎麼寫) ——但只教語法本身**，List Report / Object Page 真正的版面配置邏輯、Draft 編輯體驗這些完整的畫面設計技巧，留給 Fiori Elements 課程深入。

RAP 的五層架構



每一層各司其職：Table 只負責存資料；Interface View 負責決定資料模型長什麼樣（欄位、關聯）；Behavior Definition 負責決定這個資料物件的「行為」（能做什麼操作、有什麼規則）；Service Definition 負責決定要把哪幾個 View 打包成一個服務；Service Binding 負責決定這個服務要用哪個協定版本（OData V2/V4）真正對外發布。這五層在 rap02 ~ rap04 會逐一深入，這裡先建立整體概念。

Managed 還是 Unmanaged？

RAP 的 Behavior Definition 有兩種實作模式：

- **Managed**：CRUD 操作的實際資料庫存取（**INSERT/UPDATE/DELETE**）由框架自動產生，你只需要宣告「這個實體支援哪些操作」，不用自己寫 **MODIFY zrapnn_xxx FROM ...** 這種底層程式碼。新設計的資料物件，SAP 建議優先用這個模式，語法上也是官方教材的主流。
- **Unmanaged**：你自己寫一個 ABAP 類別實作所有 CRUD 邏輯（**implementation unmanaged in class ...**），框架只負責派發呼叫、不管你怎麼存資料。這個模式通常用在「要包一層 RAP 介面在既有的舊系統/BAPI 之上」的情境——這個系統裡的標準物件 **C_SalesOrderManage**（銷售訂單管理，可以自己用 **sap-adt** 讀 **/sap/bc/adt/bo/behaviordefinitions/c_salesordermanage/source/main** 看原始碼）就是典型例子：SD 模組的銷售訂單邏輯早就存在幾十年了（**VA01/VA02** 背後那一整套），RAP 化的時候沒有重寫，而是寫一個 **implementation unmanaged in class cl_sd_behv_salesordermanage unique** 把舊邏輯包起來對外變成 RAP 服務。

⚠ **更正（rap03 出題時發現，寫這一課當下還不知道）**：這裡原本的規劃是「本課程只教 Managed，Unmanaged 太進階不教」——但 rap03 實測發現這系統的 Managed Runtime（CUD 寫入）被 SAP 官方標記為尚未對外釋出，任何 Managed BDEF 的 create/update/delete 執行到底層都會 Dump（技術細節見 rap03）。所以課程從 **rap03** 起改成兩者都教：Managed 當語法知識儲備（銜接官方教材 / 未來的 ABAP Cloud RAP 課程），Unmanaged 是這系統上真正能端對端執行、驗證的版本。Unmanaged 確實需要紮實一點的 OOP 底子（要自己實作 **LOCK/CREATE/READ** 這些方法），但既然 Managed 走不通，這是必要的取舍，詳見 rap03。

⚠ 這是 Classic RAP，不是 ABAP Cloud RAP——開課前已實測驗證的語法差異

在規劃這門課之前，我們先用一個最小的 Managed RAP BO（Table → CDS View → Managed BDEF → Service Definition，全部走 ADT / MCP 建立並成功啟用）在這個系統上做了端對端驗證，過程記錄在 [.claude/rules/sap-adt-mcp.md](#) 第 40 節。結論很清楚：這個系統的 **RAP 框架是真的在運作、不是空殼**，但屬於比較舊的世代，跟目前 SAP 官方教材 / 認證預設使用的語法有兩個具體差異：

① **沒有 View Entity 語法**：CDS 有兩種宣告根 View 的寫法。新式（2020 年之後、ABAP Cloud 世代預設）是 **define root view entity ZI_XXX as select from ... { ... }**；這個系統用這種寫法會直接在啟用時報錯：

```
Syntax error: Keyword ENTITY not allowed
```

必須改用舊式語法（拿掉 **entity** 關鍵字）：

```
@AbapCatalog.preserveKey: true
@ObjectModel.compositionRoot: true
define root view ZI_XXX
  as select from zrapnn_xxx
{
```

```
key root_id,
descr
}
```

(讀這個系統既有標準物件 `C_SalesOrderManage` 的原始碼可以看到，SAP 自己出的標準內容在這個系統上也是用這種舊式語法、以 `composition [0..*] of` 表達關聯，不是新式的 ON 條件簡化語法。)

② 沒有 **strict** 模式：BDEF 新式教材會在開頭寫 `strict ( 2 );` 宣告嚴格模式 (讓某些原本只是警告的問題變成硬性錯誤，幫助抓出設計疏漏)。這個系統完全不認得這個語法元素：

```
managed;
strict ( 2 );      " 這一行會報 "internal | with" expected, not "strict".
```

本課程的 BDEF 一律只寫最簡單的 header：

```
managed;

define behavior for ZI_XXX alias Root
persistent table zrapnn_XXX
lock master
{
    create;
    update;
    delete;
}
```

這兩個差異背後更根本的原因，是這個系統的開發套件沒有啟用「**ABAP Cloud 語言版本**」——這是 SAP 訂出的一個語言限制開關，啟用後編譯器會擋掉一大類傳統 ABAP 寫法，目的是保證程式碼能同時跑在 On-Premise 與 SAP BTP ABAP Environment (Cloud) 上：

限制項目	傳統 ABAP 可以，ABAP Cloud 不行
資料存取	只能存取已 Release 的 API，不能直接 <code>SELECT</code> 未 Release 的自訂表或標準表
畫面	沒有 Classical Dynpro，不能 <code>CALL SCREEN</code>
程序	不能用 <code>FORM/PERFORM</code> ，一律用 Class/Method
記憶體	不能用 <code>EXPORT/IMPORT TO MEMORY</code>
系統存取	不能 <code>SET CLIENT</code> ，不能直接呼叫未 Release 的 FM
權限	傳統 <code>AUTHORITY-CHECK</code> 受限，要走 Released 的權限檢查機制

ABAP Cloud 語言版本要 S/4HANA **2022 以後** 的 On-Premise 系統，或 SAP BTP ABAP Environment (Cloud) 才能啟用——這個系統的套件沒有這個開關，所以連帶地新式 CDS View Entity / BDEF strict 語法也用不了 (這兩者是 ABAP Cloud 世代一起引進的語言特性)。

RAP 在 On-Premise 版本的演進歷程 (2026-08-18 查證，用官方逐版語法對照表取代網路二手轉述)

網路上流傳一個版本演進說法：1709/1809 沒有 RAP；1909 RAP 初代只支援 Unmanaged Non-Draft，完全不支援 Managed / Draft；2020 才引進 Managed 與 Draft 基礎；2021 起 Unmanaged + Draft 才算真正成熟。查證後找到最權威的第一手資料——官方 ABAP Keyword Documentation 的 [ABENRAP_FEATURE_TABLE](#)，這份文件逐一列出**每一個 RAP BDL 語法元素**在四個維度的導入版本：ABAP Cloud 季度版號、**ABAP Release On-Premise 版號**、SAP BTP ABAP Environment、SAP S/4HANA Cloud Public Edition。對照這系統已確認的版本 ([SAP_BASIS 754](#) = On-Premise 7.54 = **S/4HANA 1909**)，可以把常見的版本號換算成年份：7.53≈1809、7.54=1909、7.55≈2020、7.56≈2021、7.57≈2022、7.58≈2023 (逐年遞增 0.01，這個對照關係本身沒有官方單一頁面直接列出，是用「這系統確認是 7.54=1909」這個錨點反推的，屬於合理推論不是官方逐字聲明)。

查表結果，需要更正的地方比原本更多：

語法元素	On-Premise 版號	對應年份 (推算)	備註
<code>unmanaged</code> (基礎關鍵字)	7.53	≈1809	比 <code>managed</code> 早一版，代表 Unmanaged RAP 的起點比常見說法早一個年度版本
<code>managed</code> (基礎關鍵字)	7.54	1909	官方逐版文件白紙黑字記載，這系統就是這個版本
<code>create/update/delete</code> (CRUD)	Unmanaged 7.53 / Managed 7.54	1809 / 1909	兩種模式從各自起點就有基本 CRUD
<code>with draft / draft table</code> (Draft 機制本身)	7.55	≈2020	官方表格記載 Draft 不是 1909 就有，是下一版才加入
<code>with additional save / with unmanaged save</code> (Managed 存檔策略)	7.55	≈2020	
<code>field(mandatory:create) / field(readonly:update)</code> (欄位動態限定語法)	7.55	≈2020	這正是本課程 rap09 加碼段落實測不支援的那個語法——跟這系統版本推算一致：7.54 這系統理論上不該有這個語法
<code>authorization master(global) / lock:none / strict</code> (一般模式)	7.56	≈2021	
<code>strict(2)</code> (官方建議的版本)	7.57	≈2022	
Late Numbering：Unmanaged BO with Draft 與 Managed BO	7.57	≈2022	官方表格特別把「Unmanaged 不帶 Draft」(7.53) 跟「Unmanaged 帶 Draft」(7.57) 分兩欄列，差了整整 4 個版本

語法元素	On-Premise 版號	對應年份 (推算)	備註
RAP Extensibility (<code>extensible</code>) / <code>with full data</code>	7.57	≈2022	
Side Effects (<code>side effects { ... }</code> · 欄位聯動更新)	7.58	≈2023	

對使用者原本兩個問題的回答：

- **Unmanaged BO** 要有「完整功能」(含 **Draft**、正確的 **Late Numbering**、**Side Effects** 這類真正影響 **Fiori Elements** 互動體驗的機制)，落在 **On-Premise 7.57 ~ 7.58 ≈ S/4HANA 2022 ~ 2023**，比網路上常說的「2021 起成熟」還要晚一到兩個年度版本——關鍵卡點是「Unmanaged BO with Draft」的 **Late Numbering** 要到 7.57 才有官方支援 (7.53 ~ 7.56 之間雖然 Unmanaged+Draft 語法可能已經能寫，但編號機制不完整)，**Side Effects** 更是要等到 7.58。
- **Managed BO** 要有「完整功能」(含 **strict(2)** 官方建議的嚴格模式、完整 **Extensibility**、**with full data** 存檔策略)，落在 **On-Premise 7.57 ≈ S/4HANA 2022**；如果只要「基本能動」(**CRUD**、**Association**、**Action**、**Validation**、**Determination**)，**On-Premise 7.54 ≈ S/4HANA 1909** 就有了 (跟先前版本已經更正過的結論一致——**managed** 語法本身從 1909 就存在)。

⚠ 這系統本身有一個目前無法解釋的矛盾，如實記錄：官方表格說 `with draft` 要 **On-Premise 7.55** 才有，但這系統確認是 7.54——照理說 `with draft`；這行應該編譯不過。可是本課程用暫時性驗證物件實測 (`managed implementation in class ... unique; with draft; define behavior for ...`)，`checkruns` 完全沒有對 `with draft`；這一行報錯 (跟這系統對 `strict / readonly:update` 這些真正不支援的語法會給出的明確 token 錯誤完全不同)。可能的解釋：這系統的 **Support Package** 可能對 **BDL** 剖析器做過零星回補 (**Patch** 有時會把下一版的個別語法元素提前開放，不代表整個版本的功能集都跟著補齊)；也可能是官方表格本身有誤差。**沒有進一步查證的管道** (**Claude** 沒有 **S-user** 帳號查不到這系統精確的 **SP** 清單)，如實記錄這個矛盾，不強行給出結論。不影響本課程的教學安排——不管 `with draft` 語法能不能編譯，**Managed CUD** 執行期一律被 `CL_CSP_MD_METADATA_FACTORY` 白名單擋住 (見 rap03)，**Unmanaged Non-Draft** 依然是這系統唯一能端對端驗證的路線，這門課也從未測過 **Draft** (不管 **Managed** 或 **Unmanaged**) 在這系統上的實際執行結果。

為什麼只列 **On-Premise**，沒有列 **Cloud (Private/Public)**：`ABENRAP_FEATURE_TABLE` 其實同時列了四個維度 (上面只挑了 **On-Premise** 這欄講)，**Cloud** 那兩欄 (**SAP BTP ABAP Environment / S/4HANA Cloud Public Edition**) 用的是**季度發布代碼** (`YYMM` 格式，例如 `2208` = 2022 年 8 月)，不是年度版本號，而且這兩個 **Cloud** 版本幾乎每次都同步拿到同一個功能 (代表 **ABAP Cloud** 語言版本是共用同一套底層基礎設施)。之所以只講 **On-Premise**，單純是因為這個專案 / 這門課連的系統就是 **On-Premise S/4HANA (SAP_BASIS 754)**，**Cloud** 版本的時程對這個系統沒有直接意義；但既然官方表格本來就有 **Cloud** 資料，這裡補上對照——**Cloud 版本幾乎每個功能都比對應的 On-Premise 版本早 4 ~ 9 個月拿到** (例如 `strict(2)`：Cloud 是 `2208` = 2022 年 8 月，**On-Premise 7.57** 對應的 **S/4HANA 2022** 是同年稍晚才發行；**Side Effects**：Cloud `2302` = 2023 年 2 月，**On-Premise 7.58** 對應 **S/4HANA 2023** 要到年底)，這是 **SAP「Cloud-first」交付模式**的典型現象——新語言特性先在 **Cloud (BTP ABAP Environment / S/4HANA Cloud)** 上線驗證，隔幾個月才打包進下一個 **On-Premise** 年度版本。如果之後要接上一個真正的 **ABAP Cloud** 語言版本系統 (rap01 前面提過的規劃)，到時候應該直接查 **Cloud** 這兩欄的版號，不能沿用這裡整理的 **On-Premise** 對照。

Deep Parameter (Abstract Entity 作為 Action 的巢狀輸入結構) 版本演進 (2026-08-21 查證 + 實測 , rap10)

網路上流傳一個說法：Deep Parameter (用 CDS Abstract Entity 當 Action 的巢狀輸入 , 能一次傳整包 Header+Item 的深層 JSON) 在 S/4HANA 2020 開始「部分 / 試行支援」, 2021 才「完全成熟」, 1909 則「完全不支援」。查 [ABENRAP_FEATURE_TABLE](#) 這份官方逐版語法對照表 (同一份第 102 節已經用過的權威來源) 核對 , 結論方向對、細節要更正：

語法元素	On-Premise 版號	對應年份 (推算)
<code>abstract</code> (定義 Abstract BDEF 本身)	7.56	≈2021
<code>with hierarchy</code> (Abstract BDEF Header , 深層巢狀衍生型別的開關)	7.56	≈2021
<code>deep [table] (parameter)</code> (Action / Function 的深層輸入參數)	7.56	≈2021
<code>deep mapping for ...</code> (Abstract BDEF 內的深層欄位映射)	7.56	≈2021
<code>association ... [with hierarchy]</code> (Abstract BDEF 內的巢狀關聯)	7.56	≈2021

跟網路說法的差異：官方表格顯示的是單一版本 (7.56≈2021) 乾淨切換——`abstract` / `with hierarchy` / `deep parameter` / `deep mapping` 這一整組構成 Deep Parameter 機制的語法元素 , 全部同時在 7.56 才出現 , 7.56 之前 (含 7.55≈2020) 完全沒有 , 沒有官方記載的「2020 部分 / 試行支援」這個中間階段。「1909 完全不支援」這個結論是對的 , 但不是因為 2020 有「部分」支援、2021 才「完全」成熟 , 而是 **7.55 / 2020** 一樣是完全沒有 , **7.56 / 2021** 才整組一次到位。

這系統 (7.54 = 1909) 的實測結果 , 跟官方表格完全吻合：本課程 rap10 出題前查證階段 , 在 `$TMP` 建了一組 Abstract Entity + Composition (`ZA_RAP10_HDR` / `ZA_RAP10_DTL`) 測試 , 結果分兩層：

1. **CDS 層** (`define abstract entity + composition/association to parent`) 本身可以編譯、啟用成功——這點比較意外 , 代表 CDS DDL 層的 Abstract Entity / Composition 語法比 BDL (Behavior Definition Language) 層寬鬆 , 這系統的 CDS 編譯器已經認得這些關鍵字。
2. 但 **Abstract BDEF** 的 `with hierarchy` 這一行直接編譯錯誤："`BOPF | draft`" expected, not "`hierarchy`".——證實這系統的 BDL 剖析器 (跟 CDS DDL 剖析器是分開的元件) 版本比 7.56 舊 , 合法的 `with` 後綴詞彙裡根本沒有 `hierarchy` 這個選項。
3. **Action** 定義裡的 `deep` / `deep table` 關鍵字 , 剖析器直接不認得："`static action X deep table parameter Y; 報 "; | external | parameter | result`" expected, not "`deep`".——連「先允許語法、等執行期才報錯」的機會都沒有 , 這是剖析階段就被拒絕。

結論：Deep Parameter 在這系統 (On-Premise 7.54 / S/4HANA 1909) 完全不可用 , 不管是 CDS 層或 BDL 層都卡關 (BDL 層是硬性卡關 , CDS 層雖然能編譯但沒有 BDL 端點可用也是白搭)。Workaround (rap10 採用的做法)：不建 Abstract Entity , 改成讓 Action 的扁平 (flat) `parameter` 直接引用一個既有的 **classic DDIC 巢狀型別** (結構裡的某個欄位本身是 Table Type , 例如 `header : zsqm005_inb_h; detail : ztt_qm005_inb_d;` 這種原生 ABAP 巢狀結構)——`parameter` 子句本身從 Unmanaged 7.53 / Managed 7.54 就存在 (見上面主表) , 沒有版本限制 , 且官方文件明講「輸入參數可以是 CDS Abstract Entity , 也可以是 classic DDIC type」, 兩者都合法。這代表：只要巢狀資料結構已經用傳統 **ABAP Structure/Table Type** (不透

過 RAP 自己的 Composition 機制) 事先定義好 , Action 完全可以用「一個參數扛住整包深層資料」 , 不需要 Deep Parameter 這整組 7.56+ 才有的機制——差別只在於**型別檢查與 EML %control / %param-%target** 這類 **RAP 原生語法糖拿不到** , 但對「純粹把資料整包塞進 Action、內部邏輯自己解析」這種情境完全夠用。詳細案例見 [src/ABAP_Training_RAP/rap10_legacy_integration.md](#)。

✅✅ **Cloud 對照組驗證 (2026-08-21 , 真正的 ABAP Cloud / BTP Trial , SAP_BASIS 版號比這系統新)** : 上面的版本落差判斷已經用真正的 Cloud 系統驗證成立——**with hierarchy / deep table parameter** 在 Cloud 上完全能編譯、啟用 , 且用 ABAP Unit Test 端到端驗證過巢狀資料真的能從 EML 呼叫端傳遞到 Action Handler (不只是編譯過關) 。但緊接著發現一個比版本落差更根本的限制 : **deep table parameter** 完全沒辦法透過 **OData** 曝露 , 不分 **V2 / V4**——建 Service Binding 不管走 OData V2 還是明確選 OData V4 , Cloud 的編譯器都在 Service Binding 建立階段直接拒絕 : 「**Action SUBMIT: Deep table parameters are not supported for OData exposure**」。這代表即使換到語法完整、版本夠新的系統 , **Deep Parameter** 機制本質上也只是給 **ABAP 端 EML** 消費者用的 , 不是設計給 **OData / Fiori** 這種外部協定消費者用的——rap10 選擇「扁平 Parameter + Wire Adapter 自己解析 JSON」這條路 , 不是受限於系統版本才退而求其次的 workaround , 而是唯一能讓外部 **HTTP 呼叫端 (Postman、Fiori)** 碰得到巢狀資料的架構 , 換到任何版本的系統結論都一樣。完整過程見 [rap10_legacy_integration.md](#) 「Cloud 對照組」段落。

RAP BO (Business Object) 這個詞是什麼 ?

這門課從 rap01 開始就一直用「BO」這個縮寫 (Managed BO / Unmanaged BO / RAP BO) , 這裡正式說明 : **BO = Business Object (業務物件)** , 在 **RAP** 的語境下全稱是 **RAP Business Object (RAP BO)** , 是官方 ABAP Keyword Documentation 定義的正式詞彙 ([ABENRAP_BO_GLOSRY](#)) 。

一個 RAP BO 代表現實世界的一個業務實體 (例如「訂單」「客戶」「產品」) , 主要由一份 **Behavior Definition (BDEF)** 描述——BDEF 針對一組階層式排列的 CDS Entity (Root Entity + Child Entity , 透過 Composition 串起來 , 就是這門課一路在講的 Table→CDS View→BDEF 那個結構) , 定義它的 **RAP BO Operations** (CRUD、Action、Function 這些能做的操作) 跟 **Behavior Characteristics** (欄位限定、鎖定、權限這些規則) 。RAP BO 的資料在執行期存在一個叫 **Transactional Buffer** (交易緩衝區) 的暫存區 , 直到 **COMMIT ENTITIES** 才真正寫回資料庫。

官方文件把 RAP BO 分成三種 (依「Transactional Buffer 由誰提供」區分) :

- **Managed RAP BO** : Buffer 全部或部分由框架自動提供 (**Managed RAP BO Provider**) ——這就是這門課 Part A 教的 **managed** ; 語法 , CRUD 不用自己寫底層存取邏輯。
- **Unmanaged RAP BO** : Buffer 由開發者自己在 **ABAP Behavior Implementation** (就是這門課的 **lhc_header/lhc_item** 這些 Local Handler 類別) 裡提供——這就是這系統上真正能跑的路線。
- **BOPF-based RAP BO** : 從既有的 CDS-based BOPF Business Object (比 RAP 更早的框架) 遷移過來的 , 不能從零開始新建 , 這門課沒有涉及。

這跟你在 rap01 開頭讀到的「舊式 **ABAP Programming Model for SAP Fiori** (基於 **BOPF+CDS+SEGW**) 」的 **BOPF** 是同一個概念家族——BOPF (Business Object Processing Framework) 本身也有自己的「Business Object」概念 , RAP 是 BOPF 之後的下一代框架 , 兩者的「BO」在精神上一脈相承 (都是「用宣告式方式定義一個業務實體的資料+行為」) , 但技術實作完全不同 (RAP BO 靠 CDS+BDEF+EML , BOPF BO 靠自己的一套 Node/Association/Action Repository API) , 不能直接互換概念——這也是為什麼「BOPF-based RAP BO」需要一個專門的遷移機制 , 而不是直接相容。

⚠️ **OData 服務發布 : V2 能力有限 , V4 完全不能透過 ADT 發布**

Service Binding 決定要用哪個 OData 版本對外發布。查證這個系統的 ADT discovery 文件 (/sap/bc/adt/discovery) 發現：

- **OData V2** 有完整的發布/取消發布 API (/sap/bc/adt/businessservices/odatav2/publishjobs) ——但實測呼叫這支 API 時，回報 **Metadata not loaded ... Service Definition is not available**，這是 SAP Gateway 後端的錯誤，代表除了 ADT 層之外，系統可能還需要額外的 Gateway Hub 設定 (通常是 Basis 團隊在 /IWFND/MAINT_SERVICE 或類似交易碼做的一次性系統設定)，不是靠寫程式能繞過的。
- **OData V4** 完全沒有對應的 ADT 發布 API——discovery 文件裡連 **odatav4** 這個 collection 都找不到 (雖然可以用 ADT 建立一個 V4 的 Service Binding 物件，但沒辦法讓它真正「Publish」變成可呼叫的端點)。系統裡看得到的 V4 服務 (如標準物件 C_SALESORDERMANAGE_SRV) 都是舊有標準內容，不是這個環境現在還能新建發布的。

這代表本課程的每一個練習，最後「服務真的能被 Postman/瀏覽器打到」這一步，需要你在 **SAP GUI 或 Fiori 的 RAP Business Services 應用** 手動按 **Publish**，Claude 這邊能做到「物件建立、語法檢查、成功啟用」，跟 REST 課程 (第 15 節)、Smartform 課程 (第 19 節) 的處境一樣。

OData Service 掛的路徑跟 REST 課程的自訂 SICF 服務不一樣

如果你上過 REST 課程 (src/ABAP_Training_REST/)，會發現那邊每個 Service 的網址都是 /sap/bc/zrest_training/rsNN/...，但這門課建出來的 OData Service 網址卻是 /sap/opu/odata/sap/<Service Binding 名稱>/... (V2) 或 /sap/opu/odata4/sap/.../... (V4) ——兩者完全是不同的路徑根，而且建立方式的自動化程度也不一樣：

REST 課程的自訂 SICF 服務		這門課的 OData Service
路徑根	/sap/bc/...	/sap/opu/odata/sap/... (V2) /sap/opu/odata4/sap/... (V4)
路徑命名	開發者自己取的—— zrest_training、rs01 這些節點名稱是你在 SICF 手動建立時自己打的	框架自動產生，固定公式是 /sap/opu/odata(4)/sap/<Service Binding 名稱>/...，開發者沒得自己命名這段路徑
誰負責掛 SICF 節點	開發者自己在 SICF 手動 New Sub-Element、掛 Handler Class	Service Binding 按下 Publish 時，框架自動幫你建立並啟用對應的 ICF 節點，不需要 (也不能) 自己去 SICF 手動建

為什麼要設計成這樣：/sap/bc/ 是 ICF 底下通用、完全開放給開發者自由命名的空間，BSP 網頁、自訂 REST/HTTP Handler 都掛在這裡，SAP 不干涉你怎麼組織這段路徑。但 /sap/opu/odata(4)/ 是 **SAP Gateway / OData Runtime** 框架保留的專用命名空間——因為 OData 服務需要讓 Fiori Launchpad、Service Catalog、\$metadata 探索工具用統一規則去發現/呼叫任何一個服務，路徑結構必須是可預期的固定公式，不能每個開發者各自取名字，所以這段命名空間完全由框架自動管理，不開放手動掛節點。

這也是為什麼第 40.9 節那次除錯會卡住：用 ADT REST API 手動 POST 建的 Service Binding，只做到「DDIC Repository 物件存在」，沒有觸發框架把它材料化、自動掛進 /sap/opu/odata/sap/... 這個保留命名空間底下——這一步只有 Eclipse 官方精靈的 Publish 按鈕會做，這也是為什麼手動建的版本永遠 Publish 失敗、SICF 樹狀清單裡完全看不到對應節點的根本原因。

⚠ 珍貴經驗：測試網址要用 Eclipse 開出來的，不要用 SICF 「Test Service」開出來的（2026-08-02 使用者實測發現）

Publish 成功之後，Eclipse Service Binding 編輯器的 Preview... 按鈕會自動開瀏覽器，網址類似 `https://erpdemo01.itts.com.tw:44300/sap/bc/adt/businessservices/odatav2/feap?feapParams=...`——這個網址身處外網也連得到。但如果改用傳統 SICF 交易碼對某個服務節點按右鍵 Test Service，自動開出來的網址（如 `http://s4d1909fps01.itts.com.tw:50000/...`）外網通常連不到——兩者背後的原因是：

- SICF 的「Test Service」老實地把系統自己認知的內部真實主機名稱 / Port 拼進網址，這串位址通常只有內網 / VPN 環境連得到
- Eclipse 之所以能開出正確可用的外網網址，是因為 Eclipse 本身連線這套系統時，用的本來就是對外別名 `erpdemo01.itts.com.tw:44300`（Eclipse 遠端開發本來就需要外網連得到），所以它產生的所有網址自然沿用這個對外主機名稱——不是 Eclipse 這個工具比較聰明，純粹是連線來源不同

這套系統的外網存取是透過 Reverse Proxy / SAP Web Dispatcher 之類的邊界元件轉送進來的，外部看到的主機名稱 / Port 跟系統內部 ICM 實際監聽的完全是兩回事，對應關係是 Basis / 網管設定的，不會自動同步（外部通常在邊界做 SSL Termination，所以外部走 HTTPS 如 44300，內部維持單純 HTTP 如 50000）。遇到「服務明明建好啟用卻打不通」，第一件事是檢查網址裡的主機名稱是不是內部名稱，換成 `erpdemo01.itts.com.tw:44300` 這個已知對外別名，路徑部分維持不變再測一次——這招在 REST 課程（第 15/41 節）已經驗證過同樣有效，OData / REST 服務都適用。

跟未來課程的關係

這門課教的是能在這個系統上實際跑得動的 Classic RAP 語法——這不代表白學，資料模型設計、Managed BDEF 的 CRUD / Determination / Validation / Action 這些核心概念在 ABAP Cloud RAP 裡完全一樣，只是外層語法（`view entity` 關鍵字、`strict` 子句、ABAP Cloud 限制語法）不同。等以後有機會接上一個支援 ABAP Cloud 語言版本的系統（S/4HANA 2022+ On-Premise，或另外設定一個 SAP BTP ABAP Environment 的 MCP 連線——這是另一個獨立專案目錄的事，不會混進這個 repo），會另開一門課專門教兩者的語法差異與遷移注意事項。

學習目標

- 能講出 RAP 相對於「手寫 FM/BAPI/REST Handler」這幾種舊做法解決的核心問題（減少重複性基礎設施代碼）
- 能畫出 RAP 五層架構圖（Table → Interface View → Behavior Definition → Service Definition → Service Binding），講出每一層的職責
- 能分辨 Managed / Unmanaged 兩種 Behavior Definition 實作模式的差異與各自適用情境，並能在這個系統裡找到一個 Unmanaged 的標準範例（`C_SalesOrderManage`）
- 能講出這個系統跟 ABAP Cloud RAP 官方教材的兩個具體語法差異（無 `view entity`、無 `strict`），並知道背後原因是套件沒有啟用 ABAP Cloud 語言版本
- 能講出 ABAP Cloud 限制語法大致限制了哪些傳統寫法（Released API、無 Classical Dynpro、無 FORM/PERFORM 等）
- 能講出本課程 OData 服務發布的實際限制（V2 API 存在但卡在 Gateway 設定、V4 無 ADT 發布 API），知道最後一步要靠 SAP GUI 手動操作
- 知道本課程的範圍界線：後端 OData Service + CDS View 的 `@UI.*` Annotation 語法基礎（為銜接 Fiori Elements 課程鋪路），但不包含完整的 Fiori Elements 畫面設計技巧

- 知道「BO」是 RAP Business Object 的縮寫，知道 Managed / Unmanaged / BOPF-based 三種 RAP BO 的差異來自「Transactional Buffer 由誰提供」
- 能講出 Unmanaged / Managed BO 要有「完整功能」（Draft、Late Numbering、Side Effects 等）大約要到哪個 On-Premise 版本區間（≈S/4HANA 2022 ~ 2023），並知道這個資訊查證自官方 [ABENRAP_FEATURE_TABLE](#) 逐版語法對照表，不是網路二手轉述

下一步預告

rap02 開始動手：先設計一個簡單的資料模型（DDIC Table + CDS Interface View），這是所有後續練習（Determination / Validation / Action / Associations）共同的地基；接著會加一個 Metadata Extension，教基礎的 [@UI.*](#) Annotation 語法，幫之後的 Fiori Elements 課程鋪路。